

P.PORTO

Angular Componentes & Serviços

Programação em Ambiente Web

Índice

Introdução

Components

Services

Angular Libraries

Referências

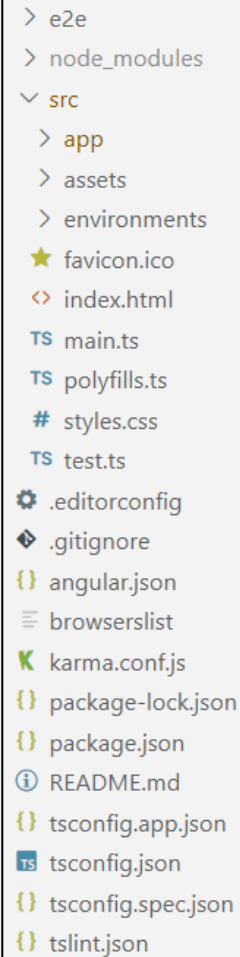
Introdução

Um projeto em Angular tem geralmente uma estrutura característica composta por vários ficheiros de configuração e do código-fonte da aplicação

Embora a estrutura seja complexa podemos apenas nos focar num subconjunto de pastas para iniciarmos as nossas primeiras aplicações

Introdução

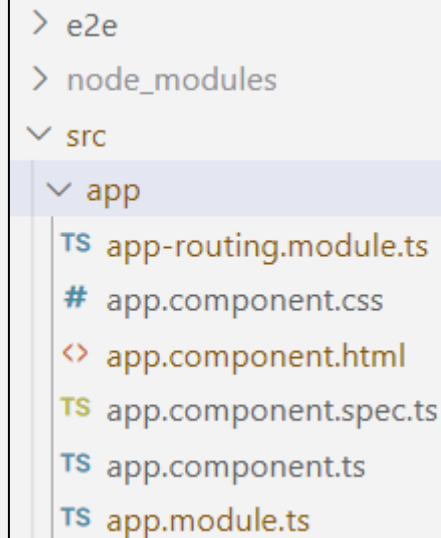
Estrutura de um projeto em Angular



```
> e2e
> node_modules
▼ src
  > app
  > assets
  > environments
  ★ favicon.ico
  <> index.html
  TS main.ts
  TS polyfills.ts
  # styles.css
  TS test.ts
  ⚙ .editorconfig
  🔍 .gitignore
  {} angular.json
  ≡ browserslist
  📄 karma.conf.js
  {} package-lock.json
  {} package.json
  ⓘ README.md
  {} tsconfig.app.json
  TS tsconfig.json
  {} tsconfig.spec.json
  {} tslint.json
```

Introdução

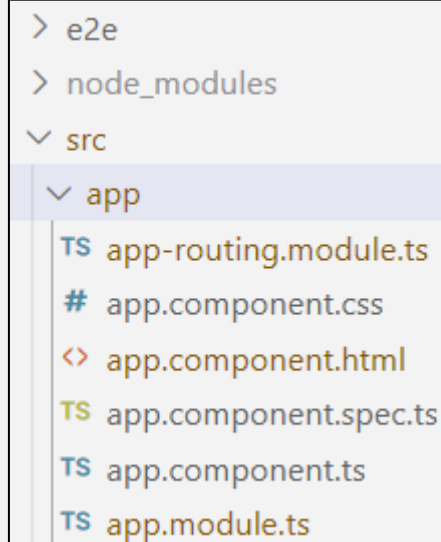
Dentro da pasta app encontram-se os principais ficheiros para o desenvolvimento da nossa aplicação



```
> e2e
> node_modules
✓ src
  ✓ app
    TS app-routing.module.ts
    # app.component.css
    <> app.component.html
    TS app.component.spec.ts
    TS app.component.ts
    TS app.module.ts
```

Introdução

Dentro da pasta app encontram-se os principais ficheiros para o desenvolvimento da nossa aplicação



```
> e2e
> node_modules
▼ src
  ▼ app
    TS app-routing.module.ts
    # app.component.css
    <> app.component.html
    TS app.component.spec.ts
    TS app.component.ts
    TS app.module.ts
```

Angular Components

Uma Framework do lado do cliente vai executar do lado do cliente executa todo o código no browser do cliente

Em angular as páginas são divididas em componentes com o padrão MVC

Angular Components

Podemos criar novos componentes com:

```
ng g component NOME_COMPONENTE
```

Cria uma pasta com 4 ficheiros

```
NOME_COMPONENTE.component.ts
```

```
NOME_COMPONENTE.component.specs.ts
```

```
NOME_COMPONENTE.component.css
```

```
NOME_COMPONENTE.component.html
```

Angular Components

Vamos assumir um componente para mostrar os detalhes de um produto

– `ng g component product-detail`

```
▼ product-detail
# product-detail.component.css
<> product-detail.component.html
TS product-detail.component.spec.ts
TS product-detail.component.ts
```

Vamos também adicionar a classe produto como um modelo de dados

```
▼ model
TS product.ts
```

```
export class Product {
  __id : String;
  name: String;
  description: String;
  quantity: Number;

  constructor(name:string, description:string, quantity:number){
    this.name = name;
    this.description = description;
    this.quantity = quantity;
  }
}
```

Angular Components

```
▼ product-detail
# product-detail.component.css
<> product-detail.component.html
TS product-detail.component.spec.ts
TS product-detail.component.ts
```

Se quisermos mostrar os detalhes do nosso produto devemos:

- adicionar a lógica do programa em product-detail-componente.ts

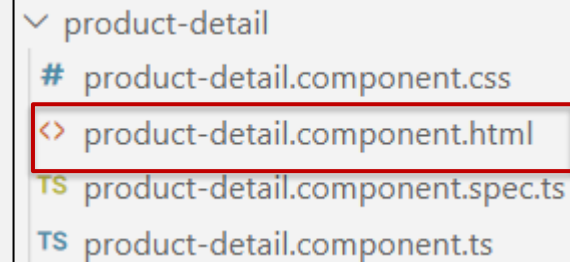
```
import { Component, OnInit } from '@angular/core';
import { Product } from '../model/product';

@Component({
  selector: 'app-product-detail',
  templateUrl: './product-detail.component.html',
  styleUrls: ['./product-detail.component.css']
})
export class ProductDetailComponent implements OnInit {
  product : Product;

  constructor() {
    this.product = new Product("shoes", "good shoes", 10);
  }

  ngOnInit(): void {
  }
}
```

Angular Components



```
▼ product-detail
# product-detail.component.css
<> product-detail.component.html
TS product-detail.component.spec.ts
TS product-detail.component.ts
```

Se quisermos mostrar os detalhes do nosso produto devemos:

- adicionar a lógica do programa em product-detail-componente.ts

```
<div *ngIf="product">
  <h2>Name</h2>
  <p> {{ product.name | uppercase }} </p>
  <h2>Description</h2>
  <p> {{ product.description }} </p>
  <h2>Quantity</h2>
  <p> {{ product.quantity }} </p>
</div>
```

Angular Components

Devemos agora criar uma rota para o nosso novo componente no ficheiro:

– app-routing.module.ts

```
const routes: Routes = [
  {
    path: 'product-detail/:id',
    component: ProductDetailComponent
  }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

Angular Components

Existem parâmetros de routing que podem ser usados para navegar entre diferentes áreas da aplicação

- ActivatedRoute – obter o link actual a seu utilizado
- Router – serviço para redireccionar o utilizador para outra página

Com estes serviços podemos obter as páginas a ser visitadas e obter parâmetros a partir do url da página

Angular Components

```
▼ product-detail
# product-detail.component.css
<> product-detail.component.html
TS product-detail.component.spec.ts
TS product-detail.component.ts
```

Modificando a página de detalhes podemos agora obter o id do produto através do url e redirecionar o utilizador para outra página

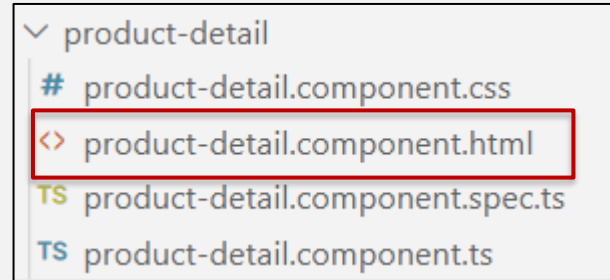
```
@Component({
  selector: 'app-product-detail',
  templateUrl: './product-detail.component.html',
  styleUrls: ['./product-detail.component.css']
})
export class ProductDetailComponent implements OnInit {
  product : Product;

  constructor( private route: ActivatedRoute, private router: Router) {
    var idTemp = this.route.snapshot.params['id'];
    this.product = new Product("shoes", "good shoes", 10);
    this.product = idTemp;
  }

  ngOnInit(): void {
  }

  seeListofProducts(){
    this.router.navigate(['/']);
  }
}
```

Angular Components



Modificando a página de detalhes podemos agora obter o id do produto através do url e redirecionar o utilizador para outra página

```
<div *ngIf="product">
  <h2>Name</h2>
  <p> {{ product.name | uppercase }} </p>
  <h2>Description</h2>
  <p> {{ product.description }} </p>
  <h2>Quantity</h2>
  <p> {{ product.quantity }} </p>
</div>
<div>
  <button (click)="seeListofProducts()" > List of Products</button>
</div>
```

Angular Components

Necessitamos agora de obter os dados a partir de uma API para usarmos produtos reais

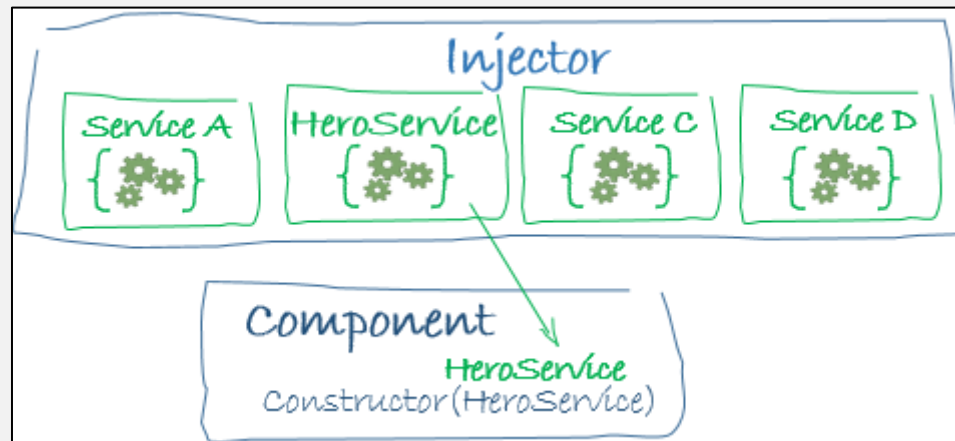
Para isso usamos a criação de um serviço em Angular

Angular Services

Os serviços são usados entre outras coisas para permitir que uma aplicação comunique com o exterior (servidores)

Podem ser usados para consumir serviços do tipo REST ou outros

Angular Services



Angular Services

Podemos criar serviços com o comando:

- `ng g service NOME_SERVICO`

Cria 2 novos ficheiros na aplicação:

- `NOME_COMPONENTE.service.ts`
- `NOME_COMPONENTE.service.specs.ts`

Angular Services

Biblioteca Rx&JS permite a Angular reagir a eventos e evitar programação síncrona

Conceitos importantes desta biblioteca que podem ser usados em serviços:

- Observables
- Subscribe de eventos

Angular Services

Adicionar um serviço para obter dados de uma API Rest

- ng g service rest

Importar biblioteca HttpClient

Executar pedidos à REST API

```
import { Injectable } from '@angular/core';
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { Observable, of } from 'rxjs';
import { Product } from '../model/product';

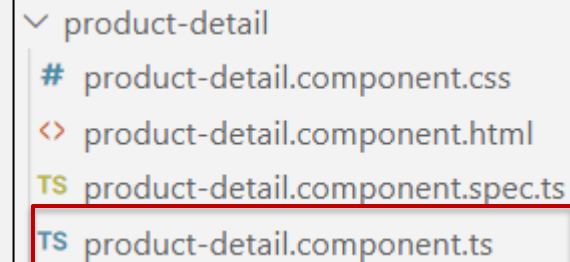
const endpoint = 'http://localhost:3000/api/v1/';
const httpOptions = {
  headers: new HttpHeaders({
    'Content-Type': 'application/json'
  })
};

@Injectable({
  providedIn: 'root'
})
export class RestService {

  constructor(private http: HttpClient) { }

  getProduct(id:String): Observable<Product> {
    return this.http.get<Product>(endpoint + 'product/' + id)
  }
}
```

Angular Components



```
▼ product-detail
# product-detail.component.css
<> product-detail.component.html
TS product-detail.component.spec.ts
TS product-detail.component.ts
```

Vamos agora obter
directamente da api
o produto e mostrar
na página

```
@Component({
  selector: 'app-product-detail',
  templateUrl: './product-detail.component.html',
  styleUrls: ['./product-detail.component.css']
})
export class ProductDetailComponent implements OnInit {
  product : Product;

  constructor(public rest:RestService, private route: ActivatedRoute, private router: Router) {

  }

  ngOnInit(): void {
    var idTemp = this.route.snapshot.params['id'];
    this.rest.getProduct(idTemp).subscribe((data : Product)=>{
      this.product = data;
    })
  }

  seeListofProducts(){
    this.router.navigate(['/']);
  }
}
```

Angular Components

Ciclo de vida do componente

Hook	Purpose and Timing
<code>ngOnChanges()</code>	Respond when Angular (re)sets data-bound input properties. The method receives a <code>SimpleChanges</code> object of current and previous property values. Called before <code>ngOnInit()</code> and whenever one or more data-bound input properties change.
<code>ngOnInit()</code>	Initialize the directive/component after Angular first displays the data-bound properties and sets the directive/component's input properties. Called <i>once</i> , after the <i>first</i> <code>ngOnChanges()</code> .
<code>ngDoCheck()</code>	Detect and act upon changes that Angular can't or won't detect on its own. Called during every change detection run, immediately after <code>ngOnChanges()</code> and <code>ngOnInit()</code> .
<code>ngAfterContentInit()</code>	Respond after Angular projects external content into the component's view / the view that a directive is in. Called <i>once</i> after the first <code>ngDoCheck()</code> .
<code>ngAfterContentChecked()</code>	Respond after Angular checks the content projected into the directive/component. Called after the <code>ngAfterContentInit()</code> and every subsequent <code>ngDoCheck()</code> .
<code>ngAfterViewInit()</code>	Respond after Angular initializes the component's views and child views / the view that a directive is in. Called <i>once</i> after the first <code>ngAfterContentChecked()</code> .
<code>ngAfterViewChecked()</code>	Respond after Angular checks the component's views and child views / the view that a directive is in. Called after the <code>ngAfterViewInit()</code> and every subsequent <code>ngAfterContentChecked()</code> .
<code>ngOnDestroy()</code>	Cleanup just before Angular destroys the directive/component. Unsubscribe Observables and detach event handlers to avoid memory leaks. Called <i>just before</i> Angular destroys the directive/component.

Angular

Necessitamos também de declarar as bibliotecas utilizadas e componentes criados no ficheiro:

– app.module.ts

```
@NgModule({  
  declarations: [  
    AppComponent,  
    HelloComponent,  
    ProductDetailComponent  
  ],  
  imports: [  
    BrowserModule,  
    AppRoutingModule,  
    BrowserModule,  
    HttpClientModule  
  ],  
  providers: [],  
  bootstrap: [AppComponent]  
})  
export class AppModule { }
```

Angular Components

Data binding entre o controlador e a view pode ser feito de várias formas e com diversos objetivos:

- One-way
- Two-way

Angular Components

Data direction	Syntax	Type
One-way from data source to view target	<pre>{{expression}} [target]="expression" bind-target="expression"</pre> 	Interpolation Property Attribute Class Style
One-way from view target to data source	<pre>(target)="statement" on-target="statement"</pre> 	Event
Two-way	<pre>[(target)]="expression" bindon-target="expression"</pre> 	Two-way

Angular Components

Type	Target	Examples
Property	Element property Component property Directive property	src/app/app.component.html <pre> <app-hero-detail [hero]="currentHero"></app-hero-detail> <div [ngClass]="{'special': isSpecial}"></div></pre>
Event	Element event Component event Directive event	src/app/app.component.html <pre><button (click)="onSave()">Save</button> <app-hero-detail (deleteRequest)="deleteHero()"></app-hero-detail> <div (myClick)="clicked=\$event" clickable>click me</div></pre>
Two-way	Event and property	src/app/app.component.html <pre><input [(ngModel)]="name"></pre>

Angular Components

Demonstração

Angular Components

Para submeter dados necessitamos de um formulário na nossa página

Para elucidar esta parte vamos criar um novo componente:

- ng g componente product-add

Angular Components

Para submeter dados
necessitamos de um formulário
na nossa página

Para elucidar esta parte vamos
criar um novo componente:

- ng g componente product-add

```
▼ product-add
# product-add.component.css
<> product-add.component.html
TS product-add.component.spec.ts
TS product-add.component.ts
```

```
@Component({
  selector: 'app-product-add',
  templateUrl: './product-add.component.html',
  styleUrls: ['./product-add.component.css']
})
export class ProductAddComponent implements OnInit {

  @Input() productData : Product = new Product();

  constructor(public rest: RestService, private route: ActivatedRoute, private router: Router) { }

  ngOnInit() {
  }

  addProduct() {
    this.rest.addProduct(this.productData).subscribe((result : Product) => {
      console.log('product added: '+result);
      this.router.navigate(['/']);
    }, (err) => {
      console.log(err);
    });
  }
}
```

Angular Components

Para submeter dados
necessitamos de um formulário
na nossa página

Para elucidar esta parte vamos
criar um novo componente:

- ng g componente product-add

```
▼ product-add
  # product-add.component.css
  <> product-add.component.html
  TS product-add.component.spec.ts
  TS product-add.component.ts
```

```
<div>
  <h2>Product Add</h2>
  <div>
    <label>Product Name:
      <input [(ngModel)]="productData.name" placeholder="Product Name"/>
    </label><br>
    <label>Product Desc:
      <input [(ngModel)]="productData.description" placeholder="Product Description"/>
    </label><br>
    <label>Product Quantity:
      <input [(ngModel)]="productData.quantity" placeholder="Product Quantity"/>
    </label><br>
  </div>
  <button (click)="addProduct()">Save</button>
</div>
```

Angular Services

Vamos adicionar a operação de add ao serviço e chamar o método respetivo da nossa API

```
const endpoint = 'http://localhost:3000/api/v1/';
const httpOptions = {
  headers: new HttpHeaders({
    'Content-Type': 'application/json'
  })
};

@Injectable({
  providedIn: 'root'
})
export class RestService {

  constructor(private http: HttpClient) { }

  getProduct(id:String): Observable<Product> {
    return this.http.get<Product>(endpoint + 'product/' + id)
  }

  addProduct (product:Product): Observable<Product> {
    return this.http.post<Product>(endpoint + 'products', JSON.stringify(product), httpOptions);
  }
}
```

Angular

Alterar o ficheiro

- app-routing.module.ts

```
const routes: Routes = [
  {
    path: 'product-detail/:id',
    component: ProductDetailComponent
  },
  {
    path: 'product-add',
    component: ProductAddComponent
  }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

Angular Components

Demonstração

Angular Component

Imaginemos agora um componente para listar produtos numa página

- ng g component product-list

```
▼ product-list
# product-list.component.css
<> product-list.component.html
TS product-list.component.spec.ts
TS product-list.component.ts
```

```
@Component({
  selector: 'app-product-list',
  templateUrl: './product-list.component.html',
  styleUrls: ['./product-list.component.css']
})
export class ProductListComponent implements OnInit {

  products: any = [];
  rest: RestService;

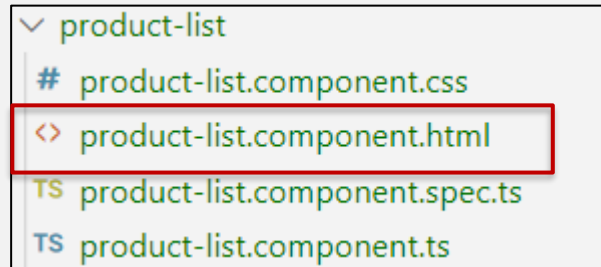
  constructor(rest: RestService, private route: ActivatedRoute, private router: Router) {
    this.rest = rest;
  }

  ngOnInit() {
    this.getProducts();
  }

  getProducts() {
    this.products = [];
    this.rest.getProducts().subscribe((data: {}) => {
      console.log(data);
      this.products = data;
    });
  }

  add() {
    this.router.navigate(['/product-add']);
  }
}
```

Angular Component



Imaginemos agora um componente para listar produtos numa página

- ng g component product-list

```
<h2>Product List</h2>

<div>
  <button (click)="add()">
    Add
  </button>
</div>

<ul class="products">
  <li *ngFor="let p of products; let i=index;">
    <a routerLink="/product-details/{{p._id}}">
      <span >{{i+1}}</span> {{p.name}}
    </a>
  </li>
</ul>
```

Angular Component

Necessitamos também de atualizar o nosso serviço com a operação para obter a lista de produtos

```
const endpoint = 'http://localhost:3000/api/v1/';
const httpOptions = {
  headers: new HttpHeaders({
    'Content-Type': 'application/json'
  })
};

@Injectable({
  providedIn: 'root'
})
export class RestService {

  constructor(private http: HttpClient) { }

  getProduct(id:String): Observable<Product> {
    return this.http.get<Product>(endpoint + 'product/' + id)
  }

  addProduct (product:Product): Observable<Product> {
    return this.http.post<Product>(endpoint + 'products', JSON.stringify(product), httpOptions);
  }

  getProducts(): Observable<Product[]> {
    return this.http.get<Product[]>(endpoint + 'products');
  }
}
```

Angular Component

Necessitamos também de atualizar a nossa tabela de routing

```
const routes: Routes = [  
  {  
    path: 'product-detail/:id',  
    component: ProductDetailComponent  
  },  
  {  
    path: 'product-add',  
    component: ProductAddComponent  
  },  
  {  
    path: 'products',  
    component: ProductListComponent  
  }  
];  
  
@NgModule({  
  imports: [RouterModule.forRoot(routes)],  
  exports: [RouterModule]  
})  
export class AppRoutingModule { }
```

Angular Components

Demonstração

Angular Components

Como seriam feitos o update e o delete usando esta abordagem simples?

Angular Libraries

Angular contém um conjunto rico de bibliotecas e componentes que podem ser importados para os nossos projetos

No entanto é também possível usar qualquer módulo ou biblioteca web javascript desde que possa ser também usada numa página simples

Angular Libraries

Adicionar framework bootstrap e jquery:

- npm install --save bootstrap jquery

Editar o ficheiro angular.json

```
"architect": {  
  "build": {  
    [...],  
    "styles": [  
      "src/styles.css",  
      "node_modules/bootstrap/dist/css/bootstrap.min.css"  
    ],  
    "scripts": [  
      "node_modules/jquery/dist/jquery.min.js",  
      "node_modules/bootstrap/dist/js/bootstrap.min.js"  
    ]  
  },  
}
```

Angular Libraries

Uma funcionalidade comum em aplicações web é a disponibilização de informação sobre mapas geográficos

Informação sobre a forma de mapas requer

- Biblioteca de mapas
- Representação da informação no mapa

Angular Libraries

Existem várias bibliotecas disponíveis para o efeito:

- Google Maps
- Leaflet
- Here Maps

As características fornecidas não variam muito e a sua escolha dependerá mais do preço de utilização e da preferência da equipa de desenvolvimento

Angular Libraries



an open-source JavaScript library
for mobile-friendly interactive maps

[Overview](#) [Tutorials](#) [Docs](#) [Download](#) [Plugins](#) [Blog](#)

Aug 21, 2018 — [Leaflet 1.3.4](#) has been released!

Angular Libraries

Podemos instalar a biblioteca leaflet com o comando:

- `ng new angular-leaflet`

Adicionar no ficheiro `angular.json`

```
"styles": [  
    "src/styles.css",  
    "src/assets/leaflet/leaflet.css"  
],  
"scripts": [  
    "src/assets/leaflet/leaflet.js"  
]
```

Angular Libraries

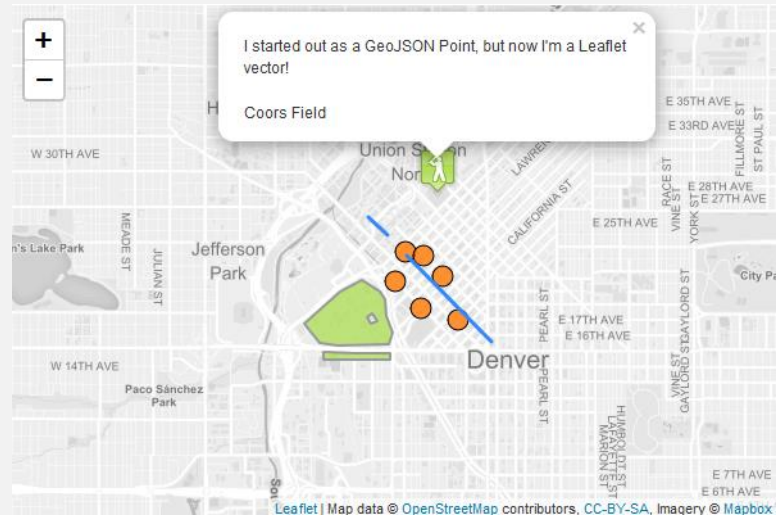
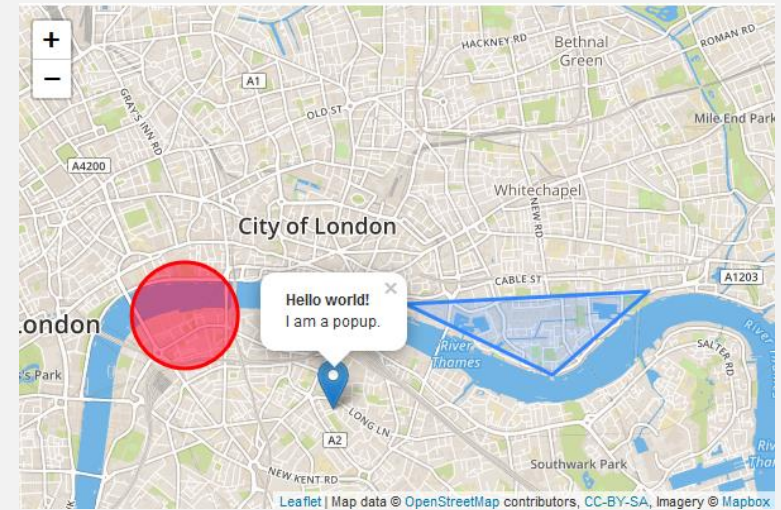
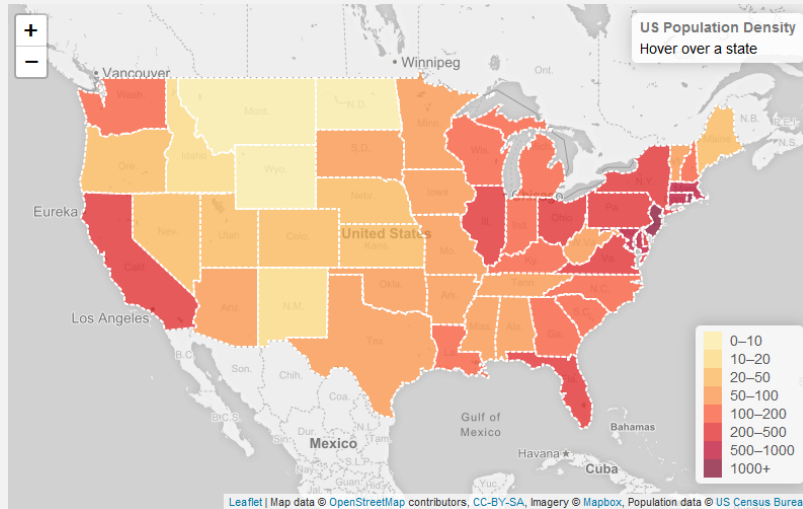
Esta biblioteca usa os mapas fornecidos pelo openStreetMaps

Para o desenvolvimento de projetos pessoais não necessita de api keys

Tem a capacidade de adicionar elementos aos mapas criados

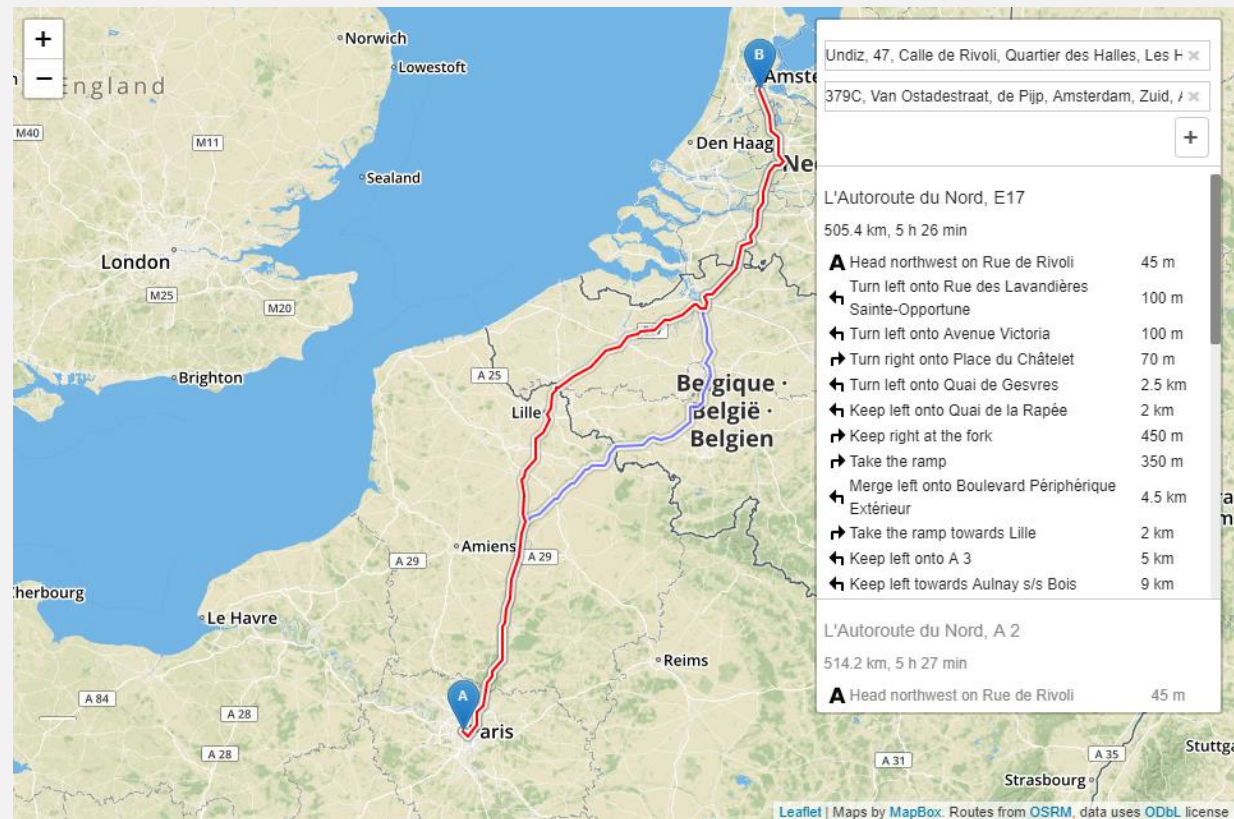


Angular Libraries



Angular Libraries

É possível
também o
cálculo de
direções e a sua
visualização no
mapa



Angular Libraries

```
import { Component, OnInit } from '@angular/core';
declare let L;

@Component({
  selector: 'app-maps',
  templateUrl: './maps.component.html',
  styleUrls: ['./maps.component.css']
})
export class MapsComponent implements OnInit {
  tmap : any;

  constructor() { }

  ngOnInit(): void {
    this.tmap = L.map('map').setView([41.371959, -8.193691], 13);

    L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
      attribution: '© <a
href="https://www.openstreetmap.org/copyright">OpenStreetMap</a> contributors'
    }).addTo(this.tmap);

  }

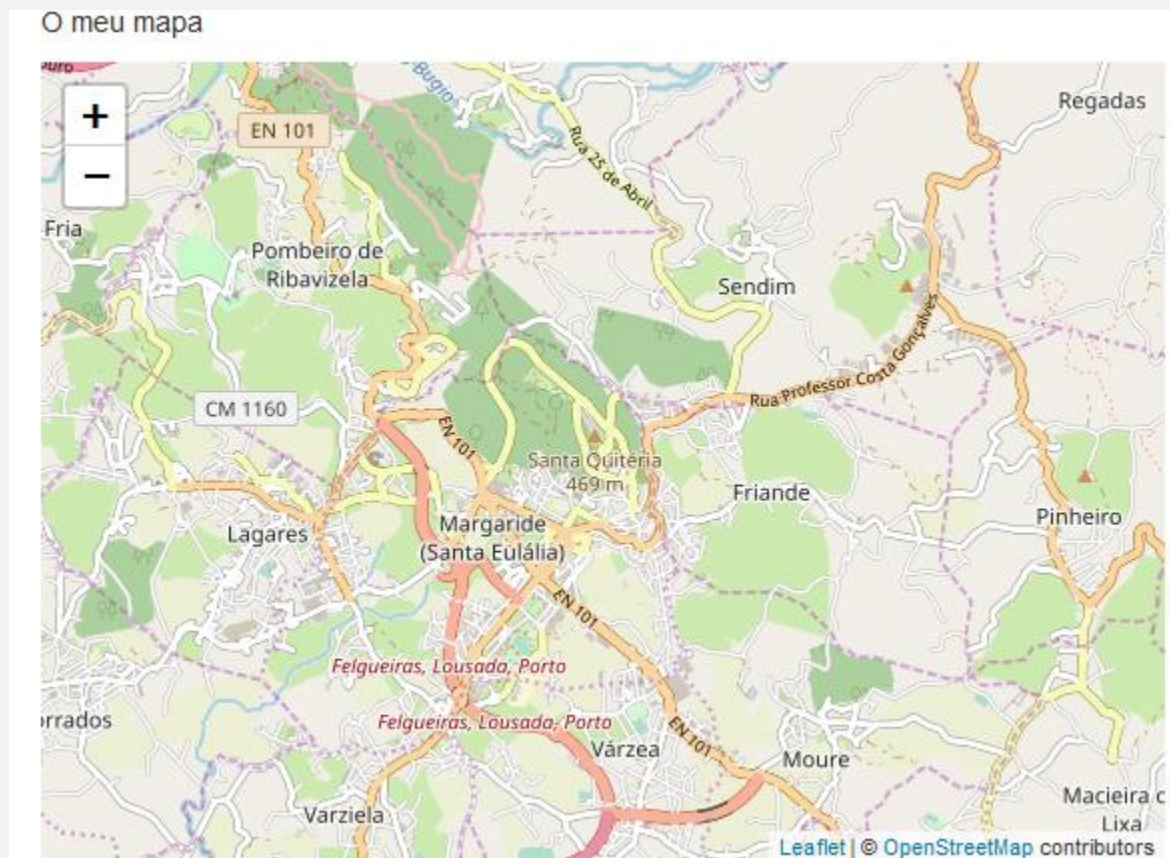
}
```

Angular Libraries

Exemplo do uso de Leaflet numa view de um componente angular

```
<p>  
O meu mapa  
</p>  
  
<div id="map"></div>
```

Angular Libraries



Referências

Angular

- <https://angular.io/>
- <https://angular.io/guide/displaying-data>
- <https://angular.io/guide/lifecycle-hooks>
- <https://angular.io/guide/observables>
- <https://angular.io/guide/http>
- <https://angular.io/guide/router>

P.PORTO

Angular Componentes & Serviços

Programação em Ambiente Web